

Yes Bank Stock Price Analysis and Next-Month Closing Price Prediction

*Exploratory analysis, statistical testing and a Linear Regression forecasting model
built on monthly data, July 2005 to November 2020*

Prepared by

Ashutosh Kumar Singh

Tools: Python, Pandas, NumPy, Matplotlib, Seaborn, mplfinance, SciPy, scikit-learn

September 2026

Abstract

This project studies the monthly share price history of Yes Bank, an Indian private-sector bank, and asks a practical forecasting question: given what is known at the end of one month, how well can the closing price of the next month be predicted? The dataset contains 185 monthly observations of the Open, High, Low and Close price from July 2005 to November 2020, with no missing values and no duplicate rows.

The work follows a standard analytics workflow. After cleaning and preparing the data, an exploratory analysis examines the long-run price trend, the relationship between the four price columns, outliers, candlestick patterns, moving averages and monthly returns. Three statistical hypothesis tests then check whether consecutive closing prices are related and whether the volatility and the average return changed after January 2018. Finally, a feature-engineered dataset is built (current prices, three lagged closes and a three-month moving average) and three models are compared on a chronological hold-out set: a naive "last close" baseline, a Linear Regression model and a Random Forest regressor.

The analysis shows that the stock rose from about 12 rupees in 2005 to a peak close of 367.90 in July 2018, then lost about 96 percent of its value by July 2020. Volatility of monthly returns rose significantly after 2018 (standard deviation 23.9 percent against 13.9 percent). On the test period, Linear Regression achieved an R-squared of 0.903 and an RMSE of 39.84, which is better than the Random Forest (RMSE 43.46) but slightly worse than the naive baseline (RMSE 38.31). The report therefore concludes, honestly, that a high R-squared on price levels is largely inherited from the persistence of prices, and that the regression model does not beat simply repeating the last close on this data. The report discusses why, lists the limitations, and proposes improvements for future work.

Keywords: Yes Bank, stock price, linear regression, time series, exploratory data analysis, hypothesis testing, Random Forest, forecasting.

Table of Contents

Abstract.....	2
1. Introduction.....	4
2. Dataset Description.....	6
3. Tools and Technologies.....	8
4. Methodology.....	9
5. Data Preparation.....	10
6. Exploratory Data Analysis.....	11
7. Statistical Hypothesis Testing.....	18
8. Feature Engineering and Model Design.....	20
9. Results and Model Evaluation.....	22
10. Discussion and Limitations.....	25
11. Using the Model for Prediction.....	26
12. Conclusion and Future Scope.....	27
References.....	28
Appendix A: Complete Source Code.....	29
Appendix B: Project Folder and Reproducibility.....	33

1. Introduction

1.1 Background

Share prices summarise the collective expectations of investors about a company. For analysts, they are a rich example of time-series data: ordered in time, strongly dependent on their own past, and occasionally hit by sudden regime changes. Yes Bank is a particularly instructive case. Its share price grew steadily for more than a decade and reached a level above 350 rupees in 2018, and it then collapsed within two years to a level close to 12 rupees. A single dataset therefore contains a long growth phase, a sharp break and a prolonged decline, which makes it a demanding test for any forecasting method.

This report documents an end-to-end analytics project on that history. It combines descriptive analysis, formal statistical testing and predictive modelling, and it is written from the code and data of the accompanying Jupyter notebook so that every number in the report can be reproduced by running the notebook.

1.2 Problem Statement

Given the Open, High, Low and Close prices of a month and the closing prices of the three preceding months, predict the closing price of the following month, and measure honestly whether the resulting model is better than a trivial benchmark.

1.3 Objectives

- Understand the structure, quality and summary statistics of the Yes Bank monthly price dataset.
- Explore price behaviour visually: trend, candlesticks, moving averages, distribution and returns.
- Test three statistical hypotheses about persistence, volatility and mean return using SciPy.
- Engineer lag and moving-average features and build a Linear Regression model for next-month closing price.
- Compare the regression against a naive baseline and a Random Forest using RMSE, MAE, R-squared and direction accuracy.
- Package the final model into a small function that returns a next-month prediction from seven inputs.

1.4 Scope

The project uses only the price columns in the supplied file. It does not use trading volume, financial statements, news, macro-economic variables or sentiment. Data are monthly, so the models predict one month ahead, not a day ahead. The work is educational and analytical; it is not investment advice.

1.5 Organisation of the Report

Chapter 2 describes the data. Chapter 3 lists the tools. Chapter 4 explains the methodology. Chapter 5 covers data preparation. Chapter 6 presents the exploratory analysis. Chapter 7 reports the hypothesis tests. Chapter 8 explains feature engineering and model design. Chapter 9 gives the results. Chapter 10

discusses findings and limitations. Chapter 11 shows how to use the model. Chapter 12 concludes and suggests future work. The appendices contain the full source code and a description of the project folder.

2. Dataset Description

2.1 Source and Structure

The data are provided in the file `data_YesBank_StockPrices.csv`. Each row is one calendar month and the file has five columns. The Date column is stored as text in the form "Jul-05" (three-letter month and two-digit year), while the four price columns are stored as decimal numbers. The file has **185 rows and 5 columns** and covers **July 2005 to November 2020**, which is 15 years and 5 months of history.

Table 1: Columns of the dataset

Column	Type in file	Meaning
Date	Text (e.g. Jul-05)	Month and year of the observation
Open	Decimal	Price at the start of the month
High	Decimal	Highest price reached during the month
Low	Decimal	Lowest price reached during the month
Close	Decimal	Price at the end of the month (the prediction target after shifting)

2.2 Summary Statistics

Table 2 shows the descriptive statistics produced by the `describe()` call in the notebook. All four price columns have almost the same mean (about 95 to 116) and a very large standard deviation (about 91 to 106). The mean is far above the median (for Close, 105.21 against 62.54), which signals a right-skewed distribution: the stock spent a long time at moderate levels and a shorter time at very high levels.

Table 2: Descriptive statistics of the price columns (185 observations each)

Statistic	Open	High	Low	Close
Mean	105.541	116.104	94.948	105.205
Std. deviation	98.880	106.333	91.219	98.583
Minimum	10.00	11.24	5.55	9.98
25th percentile	33.80	36.14	28.51	33.45
Median	62.98	72.55	58.00	62.54
75th percentile	153.00	169.19	138.35	153.30
Maximum	369.95	404.00	345.50	367.90

The highest month-end close in the file is **367.90 in July 2018** and the lowest is **9.98 in March 2009**. The highest intra-month price is 404.00 and the lowest intra-month price is 5.55, which occurs in 2020. The first close in the file (July 2005) is 12.46 and the last (November 2020) is 14.67, so the price ends the sample only slightly above where it started, even though it passed through a peak that was almost thirty times higher.

2.3 Data Quality Checks

Three quick checks were run in the notebook before any analysis. The duplicate check (`data.duplicated().sum()`) returned 0, so no month appears twice. The missing-value check (`data.isnull().sum()`) returned 0 for every column, so no imputation was needed. The data-type check showed that the price columns are already numeric, and that only the Date column needs to be converted.

Table 3: Data quality summary

Check	Result	Action needed
Rows x columns	185 x 5	None
Duplicate rows	0	None
Missing values (all columns)	0	None
Price columns numeric	Yes	None
Date column type	Text	Convert to datetime, sort, use as index

3. Tools and Technologies

The whole project is implemented in Python inside a Jupyter notebook. The libraries below are imported at the top of the notebook, and each has a specific role in the workflow.

Table 4: Libraries used and their purpose

Library	Used for
Pandas	Loading the CSV, date parsing, sorting, indexing, rolling windows, percentage change, shifting for lags
NumPy	Numerical operations such as the square root in RMSE and sign comparisons for direction accuracy
Matplotlib	Line plots, subplots and figure layout
Seaborn	Correlation heat map, box plots and the histogram with density curve; the whitegrid theme
mplfinance	The monthly candlestick chart
SciPy (stats)	Pearson correlation, Levene test for equal variances, Welch t-test
scikit-learn	LinearRegression, RandomForestRegressor, and the metrics mean_squared_error, mean_absolute_error and r2_score

Python was chosen because its analytics ecosystem covers every step of the project, from data wrangling to modelling, with mature and well-documented libraries. A notebook was used because it keeps code, output and explanation together, which suits an exploratory project.

4. Methodology

The project follows the eight-step workflow shown in Figure 1. The steps are deliberately ordered so that understanding comes before modelling: the data are cleaned and explored first, hypotheses about the behaviour of the series are tested next, and only then are features and models built.

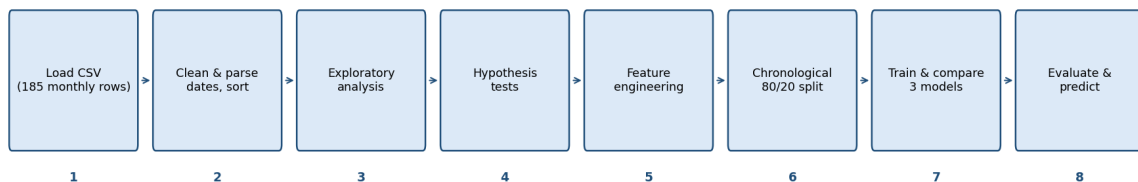


Figure 1: Overall workflow of the project

4.1 Step-by-step Description

1. **Load the data.** Read the CSV file into a Pandas DataFrame and inspect its shape, types, duplicates, missing values and descriptive statistics.
2. **Clean and prepare.** Convert the Date text into a proper datetime, sort the rows in time order and use the date as the index.
3. **Explore.** Plot price trends, compare Open, High, Low and Close, measure correlations, check outliers, draw a candlestick chart, add moving averages and study monthly returns.
4. **Test hypotheses.** Use three statistical tests at a 5 percent significance level to turn visual impressions into evidence.
5. **Engineer features.** Create lagged closing prices and a three-month moving average, and define the target as next month's close.
6. **Split chronologically.** Use the first 80 percent of months for training and the last 20 percent for testing, so the model is always tested on the future relative to its training data.
7. **Train and compare.** Fit Linear Regression and Random Forest and compare them with a naive baseline.
8. **Evaluate and predict.** Report RMSE, MAE, R-squared and direction accuracy, choose the better real model, refit it on all data and wrap it in a prediction function.

4.2 Design Principles

- **No look-ahead.** Every feature at month t uses only information available at the end of month t ; the target is the close of month $t+1$.
- **Chronological validation.** A random split would let the model train on months that come after the ones it is tested on, which overstates accuracy for time-series data.
- **A benchmark is always included.** The naive baseline that predicts next month's close as this month's close gives every model something meaningful to beat.

5. Data Preparation

5.1 Date Conversion and Indexing

Because the dates are text such as "Jul-05", they cannot be sorted or used for time-based operations. The notebook converts them with `pd.to_datetime(data["Date"], format="%b-%y")`, where %b is the abbreviated month name and %y is the two-digit year. Two-digit years in this file (05 to 20) are interpreted as 2005 to 2020. The frame is then sorted by date and the date becomes the index. The resulting period runs from 2005-07-01 to 2020-11-01, and each month is represented by its first day.

5.2 Derived Column: Monthly Return

For return analysis the notebook adds a column `Return = Close.pct_change() x 100`, the percentage change of the closing price from one month to the next. The first month has no predecessor, so there are 184 return values. Returns are more suitable than raw prices for statistical tests because prices trend, whereas returns are roughly stationary and comparable across periods of very different price levels.

5.3 Observations on the Cleaned Data

- The series is complete: every month from July 2005 to November 2020 is present exactly once (185 months).
- There are no missing values, so no rows were dropped at this stage. Later, rows are dropped only where lag and moving-average features cannot be computed (Section 8.1).
- The price scale changes enormously across the sample (from below 10 to above 400), which is important for how errors should be interpreted later.

6. Exploratory Data Analysis

6.1 Closing Price Over Time

Figure 2 shows the closing price over the whole period. The story has four phases. From 2005 to 2007 the price rises from around 12 to about 50. In 2008 it falls back to about 15 (a low of 9.98 is recorded in March 2009), and then recovers strongly through 2009 and 2010. From 2011 to mid-2018 the trend is upward, with a dip in 2013, reaching the all-time high close of 367.90 in July 2018. After that the series collapses: 343.40 in August 2018, 183.45 in September 2018, 41.45 in September 2019 and 11.95 in July 2020, a fall of about 96.8 percent from the peak.



Figure 2: Yes Bank closing price, July 2005 to November 2020

The dataset contains only prices, so it cannot explain why the price fell. The steepest monthly declines line up with periods that were widely reported as difficult for the bank (for example, regulatory actions in 2018 and 2020), but those events come from outside the data and should be verified from primary sources before being cited in any formal work.

6.2 Open, High, Low and Close Together

Figure 3 overlays the four price series. They move almost identically, and the High line is always on or above the Low line as expected. The gap between High and Low widens sharply from 2018 onwards, which shows that the price range inside a single month became much larger in relative terms, another sign of rising volatility.



Figure 3: Open, High, Low and Close prices over time

6.3 Year-wise Summary

Table 5 aggregates the monthly data by calendar year. The average closing price peaked in 2017 (315.31) and 2018 (288.64) and then fell to 125.26 in 2019 and 22.11 in 2020. Notice that the yearly low in 2008 (11.01) and 2009 (8.16) shows that an earlier deep drawdown also happened during the global financial crisis, but that episode recovered within about a year, whereas the post-2018 decline did not recover within the sample.

Table 5: Yearly summary of prices (2005 and 2020 are partial years)

Year	First open	Highest high	Lowest low	Last close	Avg. close	Months
2005	13.00	14.88	11.25	13.71	13.22	6
2006	13.68	29.66	13.40	26.97	19.35	12
2007	27.40	54.00	23.31	49.81	36.87	12
2008	49.99	55.56	11.01	15.03	28.25	12
2009	15.28	55.67	8.16	53.41	30.03	12
2010	53.60	77.60	44.60	62.54	58.65	12
2011	62.98	68.26	46.71	47.72	57.22	12
2012	47.92	95.00	46.11	92.84	74.28	12
2013	93.20	109.43	43.22	74.02	80.58	12
2014	74.74	155.00	58.42	154.57	106.95	12
2015	154.40	182.00	118.00	145.23	160.11	12
2016	145.00	290.00	126.45	231.26	213.52	12
2017	231.98	383.25	226.30	315.05	315.31	12
2018	314.00	404.00	147.00	181.75	288.64	12
2019	182.85	285.90	29.05	46.95	125.26	12
2020	46.90	87.95	5.55	14.67	22.11	11



Figure 4: Average closing price by calendar year

6.4 Correlation Analysis

The correlation heat map in Figure 5 shows that the four price columns are extremely strongly correlated, with coefficients between 0.978 and 0.995. This is expected, because all four describe the same stock in the same month. The strongest pair is Low and Close (0.995) and the weakest is Open and Close (0.978).

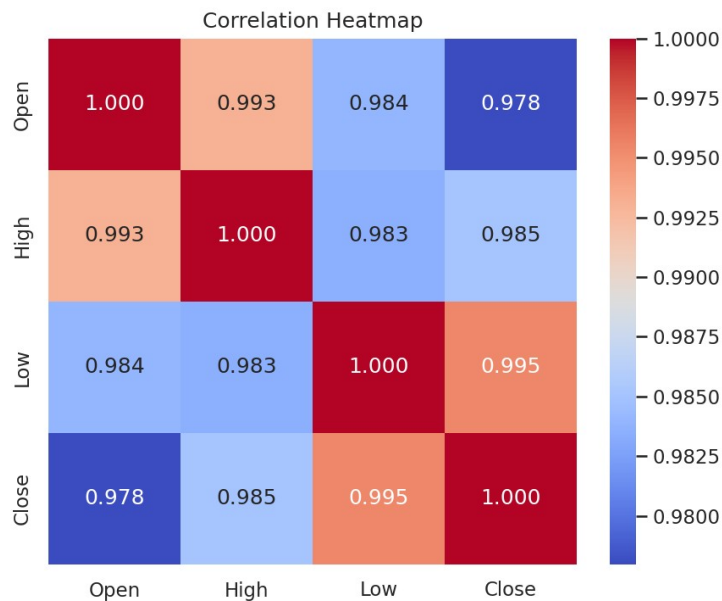


Figure 5: Correlation heat map of Open, High, Low and Close

Table 6: Correlation matrix

	Open	High	Low	Close
Open	1.000	0.993	0.984	0.978
High	0.993	1.000	0.983	0.985
Low	0.984	0.983	1.000	0.995
Close	0.978	0.985	0.995	1.000

Two consequences follow for modelling. First, any one of these columns is a good linear predictor of the others, so a regression that uses several of them will suffer from **multicollinearity**: the individual coefficients become unstable even when the overall fit is good (Section 8.4 shows this in the fitted coefficients). Second, a high correlation between price levels does not by itself mean that the model can forecast, because two trending series are almost always highly correlated.

6.5 Distribution and Outliers

The box plots in Figure 6 show, for each price column, the median, the inter-quartile range and any outliers. The distributions are right-skewed: the median (about 58 to 73) is well below the mean and the upper whisker extends to the 350 to 400 range. The long right tail comes from the 2016 to 2018 high-price period and not from data errors, so no values were removed.

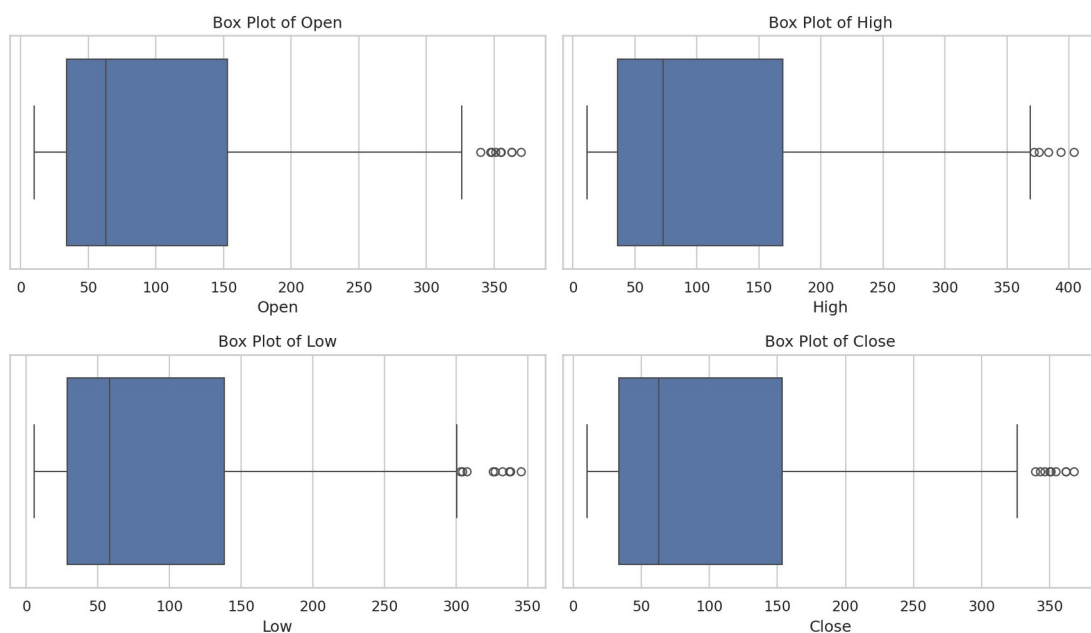


Figure 6: Box plots of Open, High, Low and Close

6.6 Candlestick Chart

A candlestick chart, drawn with `mplfinance` in Figure 7, displays Open, High, Low and Close for each month in a single glyph. The body of each candle spans open to close (one colour for a rising month, another for a falling month) and the thin lines above and below show the high and low. The chart makes the transition after mid-2018 easy to see: candles become long, with large bodies and long shadows, and are mostly of the falling colour from late 2018 to mid-2020.

Yes Bank Monthly Candlestick Chart



Figure 7: Monthly candlestick chart of Yes Bank

6.7 Moving Averages

Moving averages smooth short-term noise. Because the data are monthly, the windows are in months: 3, 6 and 12. Figure 8 shows that the short 3-month average hugs the price, while the 12-month average lags and is much smoother. During the rise to 2018 the price stays above all three averages, which is the usual signature of a healthy uptrend. In 2018 the price crosses below the shorter averages first and then below the 12-month average, and it remains below them for almost the entire decline. This lagging behaviour is also why the 3-month moving average is used as one of the model features.

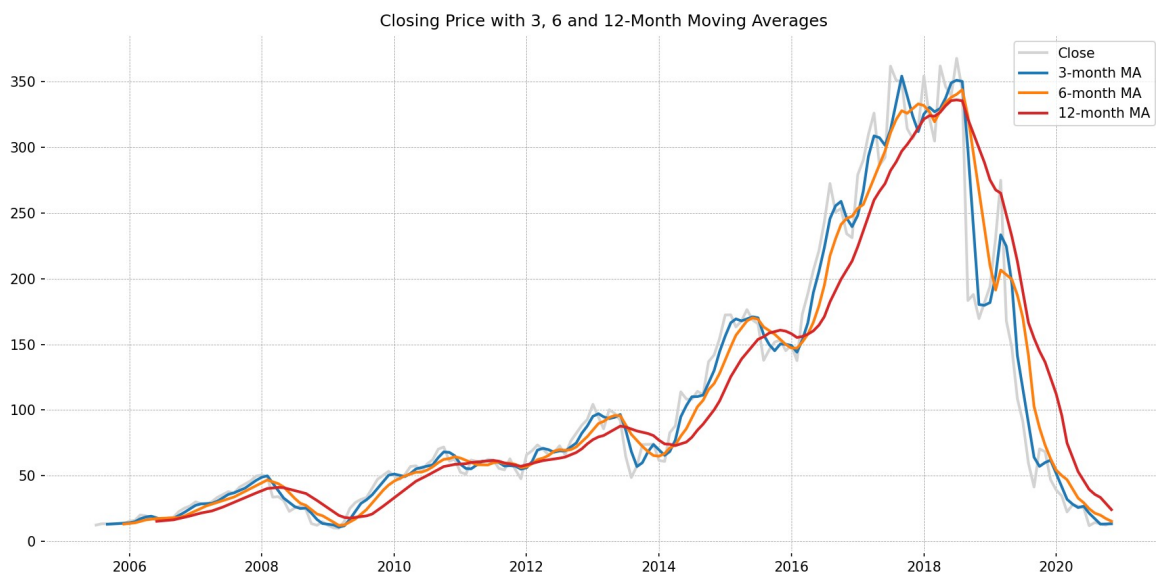


Figure 8: Closing price with 3, 6 and 12-month moving averages

6.8 Monthly Returns

The left panel of Figure 9 plots the monthly percentage return and the right panel shows its distribution. The average monthly return over the 184 months is +1.50 percent, but the standard deviation is 16.53 percent, so the typical swing is more than ten times the average drift. The best

month was October 2019 (+69.96 percent) and the worst was July 2020 (-53.32 percent). Of the 184 months, 101 were positive and 83 were negative.

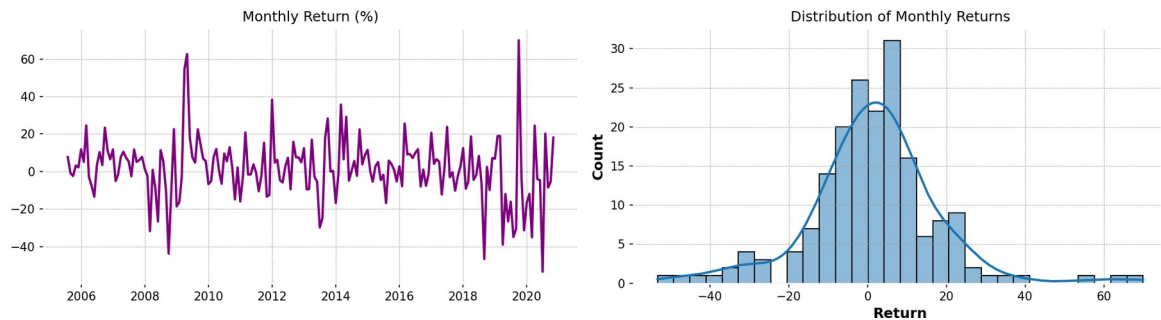


Figure 9: Monthly return over time (left) and its distribution (right)

Table 7: Monthly return statistics

Statistic	Value
Number of returns	184
Mean	+1.50 %
Standard deviation	16.53 %
Minimum (Jul 2020)	-53.32 %
Maximum (Oct 2019)	+69.96 %
Positive months / negative months	101 / 83
Skewness	0.23
Excess kurtosis	3.15

An excess kurtosis of 3.15 means the tails are much heavier than those of a normal distribution: extreme months occur far more often than a bell curve would predict. That is a warning that models built on the assumption of normally distributed errors may understate risk.

Table 8: Five largest gains and five largest losses (monthly close-to-close)

Rank	Best months	Return	Worst months	Return
1	Oct 2019	+69.96 %	Jul 2020	-53.32 %
2	May 2009	+62.59 %	Sep 2018	-46.58 %
3	Apr 2009	+54.81 %	Oct 2008	-43.72 %
4	Jan 2012	+38.29 %	Apr 2019	-38.92 %
5	Mar 2014	+35.68 %	Mar 2020	-35.02 %

6.9 Volatility by Year

Grouping the monthly returns by year gives a picture of how risky each year was. The standard deviation of monthly returns was below 6 percent in 2005 and 2007 and about 7 to 12 percent in most years from 2010 to 2017, then rose to 16.5 percent in 2018, 31.5 percent in 2019 and 23.3 percent in

2020. The only earlier years that come close are 2008 (18.8 percent) and 2009 (24.5 percent), the global financial crisis period.

Table 9: Standard deviation of monthly returns by year (%)

Year	Std. dev.	Year	Std. dev.
2005	3.9	2013	17.3
2006	11.2	2014	15.0
2007	5.6	2015	7.3
2008	18.8	2016	9.9
2009	24.5	2017	10.6
2010	8.5	2018	16.5
2011	11.1	2019	31.5
2012	12.2	2020	23.3

6.10 Summary of the Exploratory Findings

- Prices grew for more than a decade, peaked at 367.90 in July 2018 and then fell about 96.8 percent by July 2020.
- Open, High, Low and Close are almost perfectly correlated (0.978 to 0.995), so multicollinearity should be expected in regression.
- Returns are volatile (16.5 percent standard deviation) with heavy tails, and volatility was far higher in 2019 and 2020 than in 2010 to 2017.
- The series is clearly not stationary: its mean and variance change over time, which challenges any model that assumes a stable relationship between inputs and output.

7. Statistical Hypothesis Testing

To move from visual impressions to evidence, three hypothesis tests were run at a significance level of $\alpha = 0.05$. A p-value below 0.05 means that a result at least as extreme as the one observed would be unlikely if the null hypothesis were true, so the null hypothesis is rejected. The break point used in tests 2 and 3 is 1 January 2018, chosen because the price peaked in 2018 and volatility clearly changed afterwards. It splits the 184 returns into 149 months before 2018 and 35 months from 2018 onwards.

7.1 Test 1: Is This Month's Close Related to Next Month's Close?

- **Null hypothesis (H0):** there is no linear correlation between $\text{Close}(t)$ and $\text{Close}(t+1)$.
- **Alternative (H1):** a linear correlation exists.
- **Method:** Pearson correlation coefficient (`scipy.stats.pearsonr`).
- **Result:** $r = 0.978$, $p = 3.2 \times 10^{-126}$. H0 is rejected.

Consecutive closing prices are almost perfectly correlated. This justifies using past prices as inputs. It also explains why even a naive "tomorrow equals today" forecast gets a high R-squared: prices are persistent, so most of the variation in next month's price is already contained in this month's price. This test therefore shows that lag features are useful, but it does not show that the model adds forecasting skill beyond persistence.

7.2 Test 2: Did Volatility Change After January 2018?

- **H0:** the variance of monthly returns is the same before and after 2018.
- **H1:** the variances differ.
- **Method:** Levene's test for equality of variances (`scipy.stats.levene`), which is robust to non-normal data.
- **Result:** standard deviation 13.9 percent before 2018 against 23.9 percent after; $p = 0.00057$. H0 is rejected.

Volatility differs significantly between the two periods. Return risk after 2018 was roughly 1.7 times higher than before. This is a structural break: a model trained mostly on calmer years will face a market with different behaviour when it is tested.

7.3 Test 3: Did the Average Monthly Return Change After January 2018?

- **H0:** the mean monthly return is the same before and after 2018.
- **H1:** the mean returns differ.
- **Method:** Welch's two-sample t-test (`scipy.stats.ttest_ind` with `equal_var=False`), which does not assume equal variances, an important choice given the result of Test 2.
- **Result:** mean +3.12 percent before 2018 against -5.40 percent after; $t = 2.03$, $p = 0.0491$. H0 is (narrowly) rejected.

The average monthly return switched from clearly positive to clearly negative. Because the p-value of 0.049 is only just below 0.05, this conclusion is fragile: it rests on only 35 post-2018 months, and returns with heavy tails violate the normality assumption of the t-test. It is safer to describe the result as suggestive evidence of a change than as a firm proof.

Table 10: Summary of hypothesis tests ($\alpha = 0.05$)

Test	Method	Key result	p-value	Decision
1. Close(t) vs Close(t+1)	Pearson r	$r = 0.978$	$3.2e-126$	Reject H_0
2. Volatility before/after 2018	Levene	13.9 % vs 23.9 %	0.00057	Reject H_0
3. Mean return before/after 2018	Welch t-test	+3.12 % vs -5.40 %	0.0491	Reject H_0 (borderline)

Take-away. Prices are strongly persistent (Test 1), and the market regime changed after 2018 in both risk (Test 2) and average return (Test 3). These two facts, persistence and a regime change, explain most of what happens in the modelling chapters.

8. Feature Engineering and Model Design

8.1 Target and Features

The target is the closing price of the next month, created with `Close.shift(-1)`. The features are the four prices of the current month, the closing prices one, two and three months earlier (`Close_lag1`, `Close_lag2`, `Close_lag3`) and a three-month moving average of the close (`MA3`). Lags and rolling means cannot be computed for the first three months, and the target cannot be computed for the last month, so 4 rows are dropped and **181 modelling rows remain** (from October 2005 to October 2020).

Table 11: Model features

Feature	Definition
Open, High, Low, Close	Prices of the current month
Close_lag1	Closing price one month ago
Close_lag2	Closing price two months ago
Close_lag3	Closing price three months ago
MA3	Average of the last three monthly closes (including the current month)
Target	Closing price of the next month

8.2 Chronological Train-Test Split

The rows are split by time, not at random: the first 80 percent (144 months, October 2005 to September 2017) form the training set and the last 20 percent (37 months, October 2017 to October 2020) form the test set. This mirrors real use, where a model trained on the past must predict the future. It also means the test period is unusually demanding: it contains the 2018 peak, the crash and the low-price years, all at price levels that the training data (which never exceeded about 320 at month end) rarely saw.

Table 12: Train-test split

Set	Feature months	Rows	Share
Training	Oct 2005 to Sep 2017	144	80 %
Test	Oct 2017 to Oct 2020	37	20 %
Total	Oct 2005 to Oct 2020	181	100 %

8.3 The Three Models Compared

Baseline: last close

The baseline predicts that next month's close equals this month's close. It needs no training and is the standard benchmark for financial time series. A model is only useful if it beats this rule.

Linear Regression

Linear Regression fits a weighted sum of the features, $y = b_0 + b_1x_1 + \dots + b_8x_8$, by choosing the weights that minimise the sum of squared errors on the training data. It is simple, fast and

interpretable, and it can extrapolate to price levels outside the training range, which a tree-based model cannot do.

Random Forest

A Random Forest averages the predictions of many decision trees (300 in this project, with `random_state = 42` for reproducibility), each trained on a random sample of the data. It captures non-linear relationships and interactions automatically, but each tree predicts a value that is an average of training targets, so the forest cannot predict prices above the highest values it saw during training.

8.4 Fitted Linear Regression Model

Fitted on the training set, the Linear Regression has an intercept of 1.344 and the coefficients below. The coefficient of the current Close is almost exactly 1 (1.0005), which confirms that the model is mostly an "current price plus adjustment" rule. Several coefficients have signs that look odd in isolation (High is negative, `Close_lag1` is -0.653), which is the multicollinearity discussed in Section 6.4: correlated inputs share the explanatory work in ways that are unstable individually but combine to a reasonable prediction.

Table 13: Linear Regression coefficients (training set)

Feature	Coefficient
Intercept	1.3438
Open	0.4648
High	-0.1869
Low	0.1703
Close	1.0005
Close_lag1	-0.6530
Close_lag2	-0.0232
Close_lag3	0.1654
MA3	0.1081

8.5 Evaluation Metrics

- **RMSE** (root mean squared error): the square root of the average squared error, in price units. It penalises large errors heavily.
- **MAE** (mean absolute error): the average size of the error, in price units, less sensitive to a few large misses.
- **R-squared**: the share of the variance of the target explained by the model; 1 is perfect and 0 is no better than predicting the average.
- **Direction accuracy**: the share of months in which the predicted direction of change relative to the current close matches the actual direction. It measures whether the model gets the up or down call right.

9. Results and Model Evaluation

9.1 Test-set Performance

Table 15 shows the results on the 37 test months, sorted by RMSE as in the notebook. The naive baseline has the lowest RMSE (38.31) and MAE (23.36) and the highest R-squared (0.911). Linear Regression is second (RMSE 39.84, MAE 25.91, R-squared 0.903), and the Random Forest is third (RMSE 43.46, MAE 28.17, R-squared 0.885).

Table 14: Model comparison on the test set (37 months)

Model	RMSE	MAE	R-squared	Direction accuracy
Baseline (last close)	38.31	23.36	0.911	0.0 % *
Linear Regression	39.84	25.91	0.903	37.8 %
Random Forest	43.46	28.17	0.885	56.8 %

* The baseline's direction accuracy of 0 is an artefact of the calculation, not a real score: the baseline predicts no change, so its predicted difference from the current close is always zero and never matches the sign of a real up or down move. It should be ignored, or the metric should be redefined for the baseline.

9.2 Actual versus Predicted Prices

Figure 10 plots the actual next-month close together with the three sets of predictions. All three follow the actual series closely for the same reason: they are anchored to the current close. The lines also reveal the models' common weakness, which is that they react after a move instead of before it. At the 2018 peak and in the sudden fall of September 2018, all predictions stay near the previous month's high level.

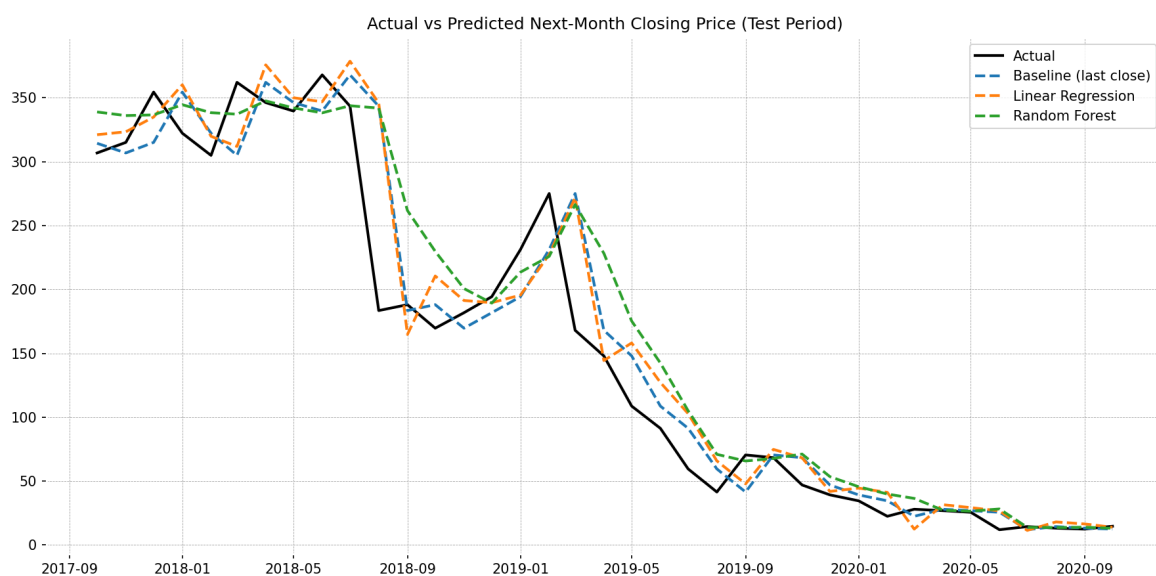


Figure 10: Actual versus predicted next-month closing price on the test period

9.3 Error Analysis

Figure 11 shows the residuals (actual minus predicted) of the Linear Regression. They are mostly negative in the second half of the test period, which means the model over-predicts during the decline. The mean error is about -12.5 (a bias towards over-prediction), and the errors are dominated by a few very large misses: the largest is about -162 for the fall from August to September 2018, followed by about -104 for the drop from March to April 2019. These two months alone account for a large share of the squared error, so RMSE is much larger than MAE.

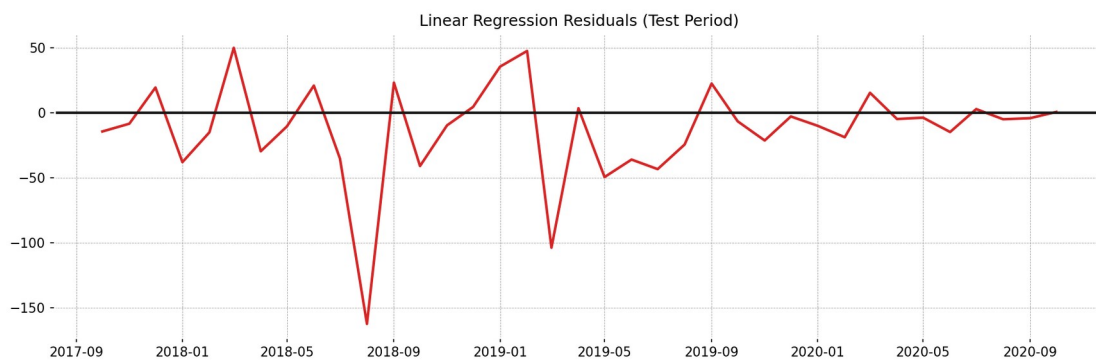


Figure 11: Residuals of the Linear Regression on the test period

Table 15: Additional error measures on the test period (not in the notebook)

Model	Mean % error (MAPE)	Mean error (bias)	Largest miss
Baseline (last close)	21.9 %	-8.1	160.0 (Sep 2018)
Linear Regression	27.8 %	-12.5	162.4 (Sep 2018)
Random Forest	28.1 %	-20.0	158.4 (Sep 2018)

The mean absolute percentage error (MAPE) is added here as a complement to the notebook metrics because RMSE and MAE are in price units and are dominated by high-price months. On a percentage basis, the typical error is 22 to 28 percent of the actual price, which is large, and the ranking of the models is the same as before: the baseline is best, then Linear Regression, then Random Forest.

9.4 Overfitting Check

The Linear Regression scores an R-squared of 0.979 on the training data and 0.903 on the test data. A drop of this size is expected when the future differs from the past, and the regime change identified in Chapter 7 is the most likely reason. The model is not badly overfitted in the classic sense (it has only nine parameters), but its training fit clearly overstates how it will perform on new market conditions.

9.5 Random Forest Feature Importance

Figure 12 shows which features the Random Forest relies on most. The current Close (0.229), the Low (0.208) and the High (0.167) carry more than half of the total importance, followed by MA3 (0.104), Close_lag1 (0.091) and Close_lag3 (0.086). The Open (0.060) and Close_lag2 (0.055) matter least. This is consistent with the linear model: what happens in the current month matters much more than the earlier months.

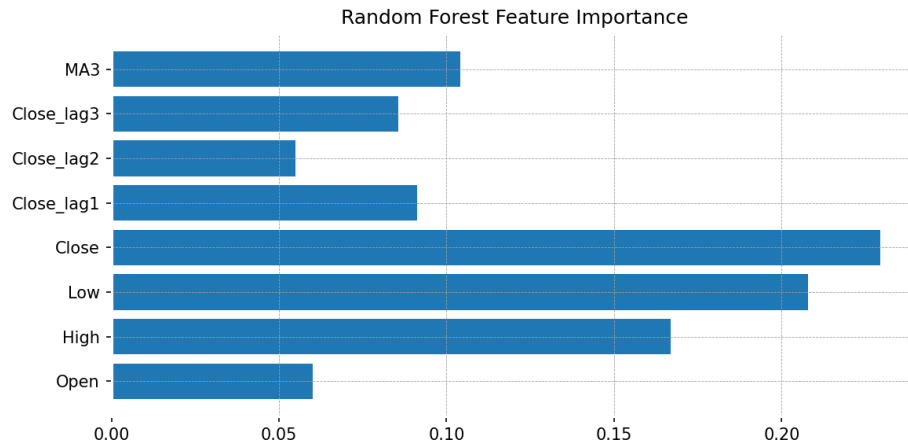


Figure 12: Random Forest feature importance

9.6 Direction Accuracy

Direction accuracy is 37.8 percent for Linear Regression (14 correct calls in 37 months) and 56.8 percent for the Random Forest (21 correct calls). The regression is therefore worse than a coin flip at calling the direction, while the Random Forest is somewhat better than a coin flip. With only 37 observations, however, a difference of this size is not statistically reliable, so it should be treated as an interesting hint rather than a conclusion.

10. Discussion and Limitations

10.1 Why Does the Simple Baseline Win?

Several reasons combine to explain why a regression with eight inputs does not beat "next month equals this month".

- **Price levels are dominated by persistence.** Test 1 showed a correlation of 0.978 between consecutive closes, so a very high R-squared is easy to obtain and does not prove forecasting skill.
- **The regime changed.** The relationship learned from 2005 to 2017, a period of mostly rising prices and moderate volatility, does not carry over to the 2018 to 2020 crash, and Test 2 and Test 3 confirm that risk and average return changed significantly.
- **The test set needs extrapolation.** The model is tested on prices as high as 367.90 while the training set contains a smaller range, and Random Forests in particular cannot predict beyond the training range.
- **Little information, few observations.** With prices alone and only 144 training months, there is very little signal beyond persistence for the model to find.

10.2 What the Project Does Show

- Linear Regression is a stronger choice than Random Forest here, which is why the notebook selects it for the final model. Its RMSE is about 8 percent lower.
- The chronological split and the baseline comparison are what make the analysis honest. Without them the R-squared of 0.90 would have looked like a successful model.
- The exploratory and statistical work gives a coherent explanation of the data: a long uptrend, a structural break in 2018 and much higher volatility afterwards.

10.3 Limitations

- **Only price information.** No volume, fundamentals, news, sector or market index data were available.
- **Small sample.** 185 monthly rows, a 37-month test set, and only 35 months after the 2018 break, so estimates are noisy.
- **Non-stationary series.** Modelling price levels rather than returns mixes very different scales, and the errors in price units are not comparable between 2010 and 2020.
- **Single split.** Results come from one train-test split; a walk-forward (rolling-origin) evaluation would give a more reliable estimate.
- **No hyperparameter tuning.** The Random Forest uses default settings apart from the number of trees.
- **Break date chosen by inspection.** The 2018 cut-off in Tests 2 and 3 was picked after looking at the data, which can make differences look stronger than a formally estimated break point would.
- **Corporate actions.** The file does not say whether prices are adjusted for corporate actions such as share issues; this should be checked before drawing firm conclusions from raw price levels.

11. Using the Model for Prediction

11.1 The Prediction Function

At the end of the notebook the better of the two real models (chosen by test RMSE) is refitted on all 181 modelling rows, and a function `predict_next_month()` is defined. It takes seven inputs: this month's Open, High, Low and Close and the closes of the previous three months (the sample notebook code also asks for these interactively). The function computes MA3 from the current close and the two previous closes and returns the predicted next-month close. In this project the selected model is **Linear Regression**.

Table 16: Inputs of `predict_next_month()`

Input	Meaning
open_, high, low, close	Prices of the current month
lag1	Close one month ago
lag2	Close two months ago
lag3	Close three months ago
(computed) MA3	$(\text{close} + \text{lag1} + \text{lag2}) / 3$

11.2 Worked Examples

Example 1: October 2020 features, target November 2020. The inputs are Open 13.30, High 14.01, Low 12.11, Close 12.42 and previous closes of 13.15, 14.37 and 11.95. The model predicts a November 2020 close of **14.94**, which compares with the actual value of 14.67, an error of 0.27 (about 1.8 percent). Note that November 2020 is part of the data on which the final model was fitted, so this is an in-sample check and not evidence of out-of-sample accuracy.

Example 2: November 2020 features, target December 2020. Using Open 12.41, High 14.90, Low 12.21, Close 14.67 and previous closes of 12.42, 13.15 and 14.37, the model returns a prediction of **17.15** for December 2020, a rise of about 16.9 percent. This value is only a demonstration of how the function works. Given the test results (typical errors of 22 to 28 percent, and worse than a naive forecast), it must not be used for trading or investment decisions.

Important. The model was not able to outperform the naive baseline on the test set, so its predictions should be read as an illustration of the method and not as a reliable forecast.

12. Conclusion and Future Scope

12.1 Conclusion

This project analysed 185 months of Yes Bank price data from July 2005 to November 2020 and built a next-month closing price model. The data were clean and complete. The exploratory analysis showed a long uptrend to a peak of 367.90 in July 2018, followed by a fall of about 96.8 percent, heavy-tailed returns and much higher volatility after 2018. Three hypothesis tests confirmed that consecutive prices are strongly correlated ($r = 0.978$), that volatility increased significantly after 2018 (23.9 percent against 13.9 percent), and, more narrowly, that the average monthly return moved from positive to negative.

In modelling, Linear Regression reached an R-squared of 0.903 and an RMSE of 39.84 on the chronological test set and was better than the Random Forest (RMSE 43.46), but it did not beat the naive last-close baseline (RMSE 38.31). The main lesson is methodological: a high R-squared on price levels is easy to get and can be misleading, and a naive baseline plus a time-ordered split are essential to judge a forecasting model fairly.

12.2 Future Scope

- **Model returns, not prices.** Predicting the percentage change removes the trend and makes errors comparable across years.
- **Walk-forward validation.** Re-train the model each month on all earlier data and test on the next month, which gives many more out-of-sample checks.
- **Time-series models.** Try ARIMA, SARIMAX or exponential smoothing, and volatility models such as GARCH, which are designed for persistent, changing-risk series.
- **Richer features.** Add trading volume, RSI, MACD, Bollinger Bands, banking-sector index returns and macro-economic indicators such as interest rates.
- **Regime awareness.** Include a regime indicator or fit separate models before and after the 2018 break.
- **Regularisation and tuning.** Use Ridge or Lasso regression to handle multicollinearity, and tune the Random Forest or try gradient boosting.
- **Dashboard.** Build a Power BI or Tableau dashboard of the price history, returns and volatility, so that non-technical users can explore the results.

References

- [1] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [2] McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*.
- [3] Virtanen, P. et al. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17, 261-272.
- [4] Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90-95.
- [5] Waskom, M. L. (2021). seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60), 3021.
- [6] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.
- [7] Hyndman, R. J., and Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts.
- [8] Levene, H. (1960). Robust tests for equality of variances. In *Contributions to Probability and Statistics*. Stanford University Press.
- [9] Welch, B. L. (1947). The generalization of Student's problem when several different population variances are involved. *Biometrika*, 34, 28-35.
- [10] Project dataset: data_YesBank_StockPrices.csv (monthly Open, High, Low, Close prices of Yes Bank, July 2005 to November 2020), supplied with the project.

Appendix A: Complete Source Code

The listing below is the code of the notebook `yes_bank_linear_regression_model.ipynb` in cell order. The pip install cells that only prepare the environment are omitted, and a cell that was repeated twice is shown once. Outputs are not shown because they are given in the main body of the report.

Cell 1

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import mplfinance as mpf
from scipy import stats
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

sns.set_theme(style="whitegrid")
```

Cell 2

```
# Load Dataset
data = pd.read_csv("data_YesBank_StockPrices.csv")
```

Cell 3

```
# Dataset First Look
data.head(180)
```

Cell 4

```
# Dataset Rows & Columns count
data.shape
```

Cell 5

```
# Dataset Info
data.info()
```

Cell 6

```
# Dataset Duplicate Value Count
data.duplicated().sum()
```

Cell 7

```
# Missing Values/Null Values Count
data.isnull().sum()
```

Cell 8

```
# Dataset Columns
data.columns
```

Cell 9

```
# Dataset Describe
data.describe()
```

Cell 10

```
# Write your code to make your dataset analysis ready.

# Assuming 'date' column is already in datetime format, if not, convert it to datetime
data['Date'] = pd.to_datetime(data['Date'], format='%b-%y')

# Confirming data types
print(data.dtypes)
```

Cell 11

```
df = data.copy()
df['Date'] = pd.to_datetime(df['Date'], format='%b-%y')
df = df.sort_values('Date').set_index('Date')
print("Period:", df.index.min().date(), "to", df.index.max().date())
df.head()
```

Cell 12

```
# Closing price over time
plt.figure(figsize=(12, 5))
plt.plot(df.index, df['Close'], color='black')
plt.title('Yes Bank Closing Price Over Time')
plt.xlabel('Date'); plt.ylabel('Closing Price')
plt.show()
```

Cell 13

```
# Open, High, Low, Close together
plt.figure(figsize=(12, 6))
for col, color in [('Open', 'blue'), ('High', 'green'), ('Low', 'red'), ('Close', 'black')]:
    plt.plot(df.index, df[col], label=col, color=color)
plt.title('Open, High, Low and Close Over Time')
plt.legend()
plt.show()
```

Cell 14

```
# Correlation heatmap
plt.figure(figsize=(6, 5))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.3f')
plt.title('Correlation Heatmap')
plt.show()
```

Cell 15

```
# Box plots
fig, axes = plt.subplots(2, 2, figsize=(12, 7))
for ax, col in zip(axes.ravel(), ['Open', 'High', 'Low', 'Close']):
    sns.boxplot(x=df[col], ax=ax)
    ax.set_title(f'Box Plot of {col}')
plt.tight_layout()
plt.show()
```

Cell 16

```
# Candlestick chart
mpf.plot(df, type='candle', style='charles', ylabel='Price',
         title='Yes Bank Monthly Candlestick Chart', figsize=(12, 6))
```

Cell 17

```
# Moving averages (monthly data, so windows are in months)
plt.figure(figsize=(12, 6))
plt.plot(df.index, df['Close'], label='Close', color='lightgray')
for window, color in [(3, 'tab:blue'), (6, 'tab:orange'), (12, 'tab:red')]:
    plt.plot(df.index, df['Close'].rolling(window).mean(), label=f'{window}-month MA',
             color=color)
plt.title('Closing Price with 3, 6 and 12-Month Moving Averages')
plt.legend()
plt.show()
```

Cell 18

```
# Monthly returns
df['Return'] = df['Close'].pct_change() * 100
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
```

```
axes[0].plot(df.index, df['Return'], color='purple')
axes[0].set_title('Monthly Return (%)')
sns.histplot(df['Return'].dropna(), bins=30, kde=True, ax=axes[1])
axes[1].set_title('Distribution of Monthly Returns')
plt.tight_layout()
plt.show()
```

Cell 19

```
alpha = 0.05
```

Cell 20

```
# Test 1: is this month's close related to next month's close?
r, p = stats.pearsonr(df['Close'].iloc[:-1], df['Close'].shift(-1).iloc[:-1])
print(f"Test 1 - Close(t) vs Close(t+1): r = {r:.3f}, p = {p:.4g}")
print(" ->", "Significant correlation" if p < alpha else "No significant correlation")
```

Cell 21

```
# Test 2: did volatility change after Jan 2018?
before = df.loc[:'2017-12-31', 'Return'].dropna()
after = df.loc['2018-01-01:', 'Return'].dropna()
stat, p = stats.levene(before, after)
print(f"\nTest 2 - Return std before vs after 2018: {before.std():.1f}% vs {after.std():.1f}%
%, p = {p:.4g}")
print(" ->", "Volatility differs significantly" if p < alpha else "No significant
difference")
```

Cell 22

```
# Test 3: did the average monthly return change after Jan 2018?
t, p = stats.ttest_ind(before, after, equal_var=False)
print(f"\nTest 3 - Mean return before vs after 2018: {before.mean():.2f}% vs
{after.mean():.2f}%, p = {p:.4g}")
print(" ->", "Mean returns differ significantly" if p < alpha else "No significant
difference")
```

Cell 23

```
model_df = df[['Open', 'High', 'Low', 'Close']].copy()
for lag in (1, 2, 3):
    model_df[f'Close_lag{lag}'] = model_df['Close'].shift(lag)
model_df['MA3'] = model_df['Close'].rolling(3).mean()
model_df['Target'] = model_df['Close'].shift(-1)      # next month's close
model_df = model_df.dropna()

X = model_df.drop(columns='Target')
y = model_df['Target']

split = int(len(model_df) * 0.8)                    # first 80% train, last 20% test
X_train, X_test = X.iloc[:split], X.iloc[split:]
y_train, y_test = y.iloc[:split], y.iloc[split:]

print("Train:", X_train.index.min().date(), "to", X_train.index.max().date(),
f"({len(X_train)} months)")
print("Test :", X_test.index.min().date(), "to", X_test.index.max().date(), f"({len(X_test)}
months)")
```

Cell 24

```
def evaluate(name, y_true, y_pred):
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
```

```

    direction = np.mean(np.sign(np.asarray(y_pred) - X_test['Close']) == np.sign(y_true -
X_test['Close']))
    return {'Model': name, 'RMSE': rmse, 'MAE': mae, 'R2': r2, 'Direction accuracy':
direction}

predictions = {'Baseline (last close)': X_test['Close'].values}

lin = LinearRegression().fit(X_train, y_train)
predictions['Linear Regression'] = lin.predict(X_test)

rf = RandomForestRegressor(n_estimators=300, random_state=42).fit(X_train, y_train)
predictions['Random Forest'] = rf.predict(X_test)

results = pd.DataFrame([evaluate(n, y_test, p) for n, p in predictions.items()])
results = results.sort_values('RMSE').reset_index(drop=True)
results.round(3)

```

Cell 25

```

plt.figure(figsize=(12, 6))
plt.plot(y_test.index, y_test, label='Actual', color='black', linewidth=2)
for name, pred in predictions.items():
    plt.plot(y_test.index, pred, label=name, linestyle='--')
plt.title("Actual vs Predicted Next-Month Closing Price (Test Period)")
plt.legend()
plt.show()

```

Cell 26

```

# Pick the best real model on the test period, then refit it on all data
best_name = 'Linear Regression' if (
    results[results['Model'] == 'Linear Regression']['RMSE'].iloc[0]
    <= results[results['Model'] == 'Random Forest']['RMSE'].iloc[0]
) else 'Random Forest'

final_model = (LinearRegression() if best_name == 'Linear Regression'
               else RandomForestRegressor(n_estimators=300, random_state=42)).fit(X, y)
print("Final model:", best_name)

def predict_next_month(open_, high, low, close, lag1, lag2, lag3):
    ma3 = (close + lag1 + lag2) / 3
    row = pd.DataFrame([[open_, high, low, close, lag1, lag2, lag3, ma3]], columns=X.columns)
    return float(final_model.predict(row)[0])

print("\nEnter this month's prices and the closes from the previous 3 months:")
labels = ["Open", "High", "Low", "Close", "Close 1 month ago", "Close 2 months ago", "Close 3
months ago"]
vals = [float(input(f"{label}: ")) for label in labels]
pred = predict_next_month(*vals)
print(f"\nPredicted next-month close: {pred:.2f} ({(pred - vals[3]) / vals[3] * 100:+.1f}%
vs this month's close)")

```

Cell 27

Appendix B: Project Folder and Reproducibility

All project material is collected in the folder Yes_Bank_Stock_Project. Its layout is shown below.

Table 17: Contents of the project folder

File or folder	Description
Yes_Bank_Project_Report.docx	This report (Word format)
Yes_Bank_Project_Report.pdf	This report (PDF copy)
yes_bank_linear_regression_model.ipynb	Original notebook with all code
data_YesBank_StockPrices.csv	Original dataset (185 monthly rows)
analysis_script.py	The notebook analysis as a plain script; regenerates figures and numbers
figures/	All charts used in the report as PNG images

How to reproduce the results

1. Install the dependencies: numpy, pandas, matplotlib, seaborn, mplfinance, scipy and scikit-learn.
2. Place data_YesBank_StockPrices.csv in the same folder as the notebook.
3. Open the notebook and run all cells in order, or run analysis_script.py to regenerate the figures and printed numbers.

The Random Forest uses `random_state = 42`, so its results are reproducible on the same library versions. Small differences in the last decimal may appear with other versions of scikit-learn.